

# 面向 AI 原生系统的智能运维

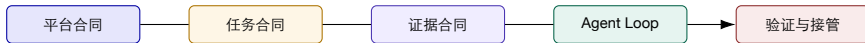
## 第 13 讲 | RCA Agent 开发实战

---

把一次故障调查变成可运行、可验证、可接管的 Agent 闭环

陈鹏飞 | 中山大学

## 从第 12 讲继续：平台合同如何落到 RCA Agent?



**第 12 讲** 定义模型、RAG、工具、状态和安全边界。

**本讲** 实现一个面向 RCA 的可运行原型。

**第 14 讲** 把 Agent 能力扩展到自然语言 ITSM 流程。

## 开场案例：模型给出“根因”，为什么还不能结束？

### 模型回答

- ▶ “可能是缓存命中下降导致请求超时”；
- ▶ 推荐重启推理服务；
- ▶ 解释流畅，置信度很高。

### Agent 必须继续问

- ▶ 哪些观测支持这个候选？
- ▶ 有没有时间先后、拓扑和变更证据？
- ▶ 哪个只读查询最能区分候选？
- ▶ 如果工具失败或证据冲突，何时停止？

本讲主线 RCA Agent 的价值不是“替人猜一个答案”，而是把调查变成证据可复核、步骤可回放、动作可控制的过程。

---

## 本讲学习目标

1. 为 RCA Agent 定义任务输入、输出、状态、预算和停止条件；
2. 设计指标、日志、Trace、拓扑、变更和知识检索工具的 schema；
3. 实现候选假设、信息增益、反证和终态验证的诊断循环；
4. 设计建议—审批—执行—回滚—验证的安全闭环；
5. 读懂 WeRCA、AgentTracer、LLMRCA、AgentChaos 和 OpsLens 的结果与边界。

完成本讲后，学生应能画出状态机，并解释一次 RCA 运行为什么停、为什么信、为什么不执行。

# 范围与非目标

## 本讲覆盖

- ▶ 任务理解与诊断计划；
- ▶ 工具契约、证据包和上下文预算；
- ▶ 候选排序、反证、停止和验证；
- ▶ 审批、回滚、记忆和故障注入。

## 不在本讲承诺

- ▶ 任意故障上的通用自动化；
- ▶ 论文数字直接等于生产 SLA；
- ▶ 没有终态验证的高风险写操作；
- ▶ 用 Prompt 长度替代系统设计。

**边界** 课程案例界面用于说明交互和状态，不代表某个产品的生产性能保证。

---

## 2 学时路线图：从任务到实验

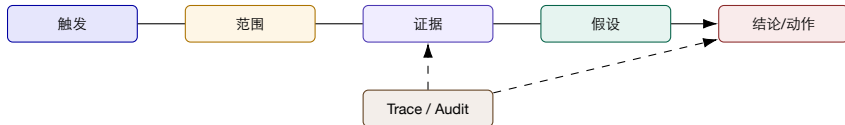
1. RCA Agent 的任务合同与端到端运行轨迹；
2. 状态机、计划、工具 schema 和证据数据面；
3. 候选假设、信息增益、反证和停止条件；
4. 审批、回滚、终态 oracle、记忆与离线演化；
5. 课程案例工作台与论文方法结果；
6. 故障注入实验、结果解释和课堂设计。

**贯穿问题** 如果只能增加一个组件来提升 RCA Agent 的可信度，你会增加证据、验证器、审批还是回放？为什么？

# 一、RCA Agent 的任务合同

先定义“完成一次诊断”是什么意思，再讨论模型如何推理

## RCA 不是一句 Prompt，而是一个任务对象



- ▶ 触发可以来自告警、工单、自然语言问题或实验注入；
- ▶ 范围包括租户、服务、集群、时间窗、影响面和权限；
- ▶ 结论必须绑定候选、证据、缺口、下一步和验证状态。



# Task Envelope：把隐含上下文显式化

| 字段                 | 示例                            | Agent 用途      |
|--------------------|-------------------------------|---------------|
| <b>incident_id</b> | inc-2026-0819-042             | 关联所有工具调用与结果   |
| <b>scope</b>       | tenant / service / cluster    | 权限过滤、拓扑边界和影响面 |
| <b>time_window</b> | [10:00,10:20]                 | 时间对齐、变更关联和回放  |
| <b>symptom</b>     | P95、5xx、TTFT、质量下降             | 生成初始假设空间      |
| <b>constraints</b> | read-only / budget / deadline | 限制计划、工具和动作    |
| <b>done_when</b>   | candidate + evidence + verify | 终态判定和人工复核     |

工程结论 没有 Task Envelope，模型只能围绕一段自然语言 “自由发挥”；有了 Envelope，平台才知道它能查什么、不能做什么。

# 输入与输出：把诊断结果变成可消费对象

## 输入

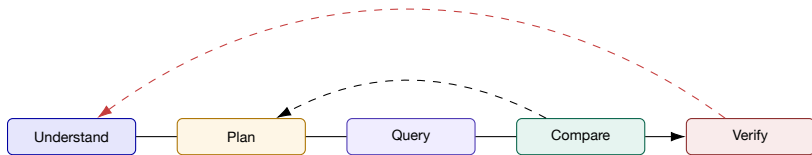
- ▶ 事件、指标、日志、Trace 和拓扑；
- ▶ 变更、版本、历史事故和运行手册；
- ▶ 任务范围、权限、预算和时限。

## 结构化输出

- ▶ 根因候选及置信等级；
- ▶ 支持证据、反证和缺口；
- ▶ 推荐下一步、风险、审批人和验证条件；
- ▶ 状态：resolved / monitor / handoff。

“自然语言解释”是最后一层呈现；内部必须先有结构化结果。

## 一次 RCA Agent 运行的最小闭环



- ▶ Plan 不是一次性生成完整步骤，而是选择下一条最有区分力的查询；
- ▶ Compare 把新证据与候选假设、时间和拓扑对齐；
- ▶ Verify 既验证结论，也验证动作是否真正改善业务结果。

---

## 任务分解：让 Agent 先缩小搜索空间

1. 事件归一化：服务、对象、时间和症状；
2. 影响面估计：用户、接口、区域和依赖；
3. 候选生成：资源、网络、版本、数据、模型和工具链；
4. 查询计划：优先低风险、高信息增益的证据；
5. 候选收敛：支持、反证、未知和冲突；
6. 结论输出：建议、审批、验证和接管。

**课堂提示** 任务分解不是把大问题切成更多 Prompt，而是把每一步的输入、输出和停止条件变得可检查。

## 置信度与执行许可必须分开



**重要** 高置信度 + 高风险仍需审批；低置信度 + 低风险可以先做只读验证。

## 端到端运行轨迹：平台真正需要记录什么？

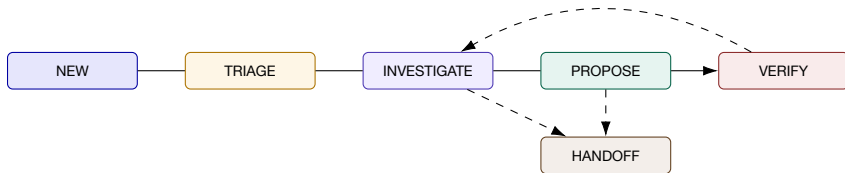
| 阶段 | 关键状态                       | 必须可回放的字段        |
|----|----------------------------|-----------------|
| 任务 | intent / scope / deadline  | 用户目标、租户、时间窗、策略  |
| 计划 | step / dependency / budget | 计划版本、选择理由、预估成本  |
| 工具 | call / result / error      | 参数、返回、耗时、权限、重试  |
| 推理 | hypothesis / belief        | 候选、支持、反证、缺口、版本  |
| 动作 | approval / execution       | 审批人、影响面、回滚、幂等键  |
| 终态 | outcome / handoff          | 业务结果、验证、接管和复盘链接 |

Agent 的 Trace 不是“模型日志”，而是一次调查的审计账本。

## 二、状态机、工具与证据数据面

把每一次查询都变成可验证、可重放的状态转移

## 状态机：每个状态都要有进入和退出条件



- ▶ 没有证据，不能从 TRIAGE 直接进入 PROPOSE；
- ▶ 验证失败时回到 INVESTIGATE 或 HANDOFF，而不是继续重复动作；
- ▶ 每次状态转移写入 Trace，便于重放和评测。



# 状态表：把“模型思考”翻译成平台契约

|                    |          |               |                                      |
|--------------------|----------|---------------|--------------------------------------|
| 状态                 | 输入前提     | 允许操作          | 退出条件                                 |
| <b>TRIAGE</b>      | 事件已归一化   | 只读范围、影响面、拓扑   | <b>scope</b> 确认或接管                   |
| <b>INVESTIGATE</b> | 候选未收敛    | 查询工具、RAG、对照检查 | 证据足够或预算耗尽                            |
| <b>PROPOSE</b>     | 有候选和证据   | 输出建议、风险、验证计划  | 审批通过或拒绝                              |
| <b>VERIFY</b>      | 动作已执行    | 健康检查、业务回归、对照  | <b>resolved / rollback / handoff</b> |
| <b>HANDOFF</b>     | 风险、冲突或未知 | 交给别人并保留上下文    | 人工接管完成                               |

结论 状态机不是限制模型能力，而是让平台知道“模型现在有资格做什么”。

---

## Run Context: 每一轮推理都要带着同一个世界模型

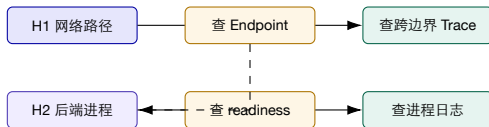
静态上下文 租户、服务、  
拓扑、权限、版本。

动态上下文 事件、窗口、  
工具结果、候选和时间水  
位。

治理上下文 预算、审批、  
脱敏、模型版本、审计标  
签。

- ▶ 每次工具调用只允许访问 Run Context 许可的对象范围；
- ▶ 结果过期、租户冲突或时间窗不一致时，标记为不可用证据；
- ▶ 上下文版本写入 Trace，避免回放时“同一个问题看到不同世界”。

## 计划不是答案：它是下一步调查的候选序列



好的计划先选择能最大区分 H1/H2 的低风险查询，而不是一次性生成十步 SOP。

---

# Plan Schema: 让计划能被程序检查

## 伪 JSON

```
{ task: "rca", step: "query_endpoint", hypothesis: "network_path",  
  tool: "topology.endpoint_list", args: {service, window},  
  expected_signal: "endpoint_empty|healthy", risk: "read",  
  next: {if: "empty", goto: "H1.support"}, budget: 1}
```

- ▶ 计划中写清“预期看到什么”，结果才有可比较的解释；
- ▶ ‘next’ 是状态转移，不是让模型自由跳转；
- ▶ 工具、参数和预算都要通过策略层验证。

# 工具注册表：描述、权限、失败语义缺一不可

| 工具                     | 输入                    | 输出                    | 关键约束      |
|------------------------|-----------------------|-----------------------|-----------|
| <b>metric.query</b>    | object、metric、window  | points、stats、quality  | 只读、时间窗上限  |
| <b>log.search</b>      | service、filter、window | hits、sample、watermark | 脱敏、结果上限   |
| <b>trace.query</b>     | trace_id、span_filter  | spans、critical path   | 租户隔离、采样标签 |
| <b>topology.lookup</b> | service、version       | nodes、edges、owner     | 当前版本、权限   |
| <b>change.lookup</b>   | object、window         | deploy、config、ticket  | 来源、时间先后   |
| <b>action.execute</b>  | plan、approval_id      | execution、rollback_id | 高风险审批、幂等  |

工具原则 工具返回空、超时、权限拒绝、部分结果和真实失败必须区分；否则 Agent 会把“没查到”误认为“没有问题”。

# 指标工具：从“查一个数”到“查一个可解释窗口”

## 请求

- ▶ 对象与标签；
- ▶ 指标与聚合；
- ▶ 时间窗与对照窗；
- ▶ 采样质量和成本预算。

## 返回

- ▶ 当前值、基线、变化点；
- ▶ P50/P95/P99 或异常分数；
- ▶ 数据水位、缺失、延迟和来源；
- ▶ 可引用的 query\_id。

同一个“延迟变高”在不同时间窗、聚合和采样质量下可能支持完全不同的候选。

# 日志工具：让 Agent 看到上下文，而不是一堆字符串

| 字段                    | 示例                   | 作用             |
|-----------------------|----------------------|----------------|
| timestamp             | 2026-08-19T10:04:22Z | 与指标、Trace、变更对齐 |
| service / pod         | checkout / pod-17    | 拓扑和对象关联        |
| severity / template   | ERROR / timeout      | 归并同类事件         |
| request_id / trace_id | req-42 / trace-7     | 跳转调用链          |
| message_redacted      | upstream reset       | 保留信息、去除敏感数据    |
| watermark / source    | 10:05 / loki         | 判断新鲜度与来源       |

工程结论 日志检索的质量不只看命中数，还看字段完整性、脱敏、时间水位和可引用性。

## Trace 工具：把一条请求拆成可以比较的路径

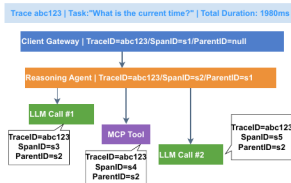


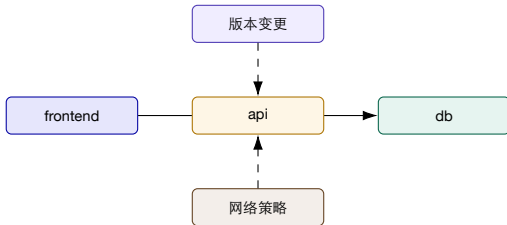
Figure 2: An example trace spanning a 3-service DMAS workflow, illustrating hierarchical span relationships and the causal decomposition of a user query into agent-level execution steps.

- ▶ 观察 root span、工具 span、模型 span 和外部服务 span；
- ▶ 比较正常样本与异常样本的关键路径、重试、等待和错误边界；
- ▶ 将 'span\_id' 绑定到证据矩阵，而不是只把整条 Trace 放进 Prompt。

来源：AgentTracer 论文材料中的独立 Trace 图；教学标注。



## 拓扑与变更工具：给候选增加结构和时间先验



- ▶ 拓扑回答“影响可能沿哪条路径传播”；
- ▶ 变更回答“候选是否在症状之前发生”；
- ▶ 二者都不是根因证明，仍需对照或干预验证。

---

## 工具失败语义：四种返回不能混为一谈

**Empty** 查询成功，但没有匹配记录。

**Partial** 部分分片、时间窗或字段缺失。

**Error** 工具或依赖真实失败。

**Denied** 权限、租户或策略拒绝。

- ▶ Empty 可能降低候选，也可能说明查询条件错误；
- ▶ Partial 必须标记缺口，不能当完整证据；
- ▶ Error/Denied 应触发重试、替代工具或人工接管，而不是继续编故事。

---

## 工具契约：把调用参数变成可测试接口

### 教学抽象

`ToolResult = {status, data, provenance, quality, cost, next_hints}`

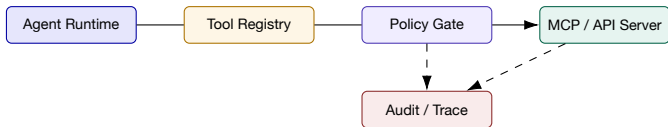
#### 必须验证

- ▶ schema 和类型；
- ▶ 对象与租户范围；
- ▶ 时间窗、分页和上限；
- ▶ 脱敏与审计标签。

#### 必须记录

- ▶ 参数摘要与版本；
- ▶ 返回状态、耗时和成本；
- ▶ 来源、时间水位和质量；
- ▶ 重试、降级和人工接管。

## MCP 与内部工具注册：协议不等于治理



**结论** MCP 或其他协议解决“怎么发现和调用”，策略层仍要解决“谁能调用、调用什么、失败如何回滚”。

---

## Evidence Bundle: 给模型的不是“所有数据”

| 部分                 | 内容                                | Agent 用法  |
|--------------------|-----------------------------------|-----------|
| <b>Observation</b> | 当前值、基线、窗口、异常                      | 描述症状和时间范围 |
| <b>Topology</b>    | 对象、依赖、版本、owner                    | 约束传播路径    |
| <b>Change</b>      | deploy、配置、策略、工单                   | 构造时间先验    |
| <b>Trace/Log</b>   | 关键 span、模板、错误边界                   | 区分候选机制    |
| <b>Knowledge</b>   | runbook、历史事故、适用条件                 | 产生可验证建议   |
| <b>Quality</b>     | freshness、coverage、partial、source | 决定证据可信等级  |

---

上下文窗口有限；证据包应可排序、可引用、可裁剪和可回放。

# 证据矩阵：每个候选都要面对支持与反证

| 候选       | 支持证据                   | 反证/缺口       | 下一步                  |
|----------|------------------------|-------------|----------------------|
| H1 网络路径  | 跨边界 span 失败、策略变更       | 直连样本正常但对照不足 | 查 Endpoint 和对照 Trace |
| H2 后端进程  | error log、readiness 下降 | 进程指标仍健康     | 查重启、资源和实例分布          |
| H3 数据/缓存 | 命中率下降、旧版本              | 业务错误不集中于读路径 | 查 TTL、版本和回放          |
| H4 负载/队列 | queue age、GPU pressure | 低负载实例也失败    | 查分区、调度和流量            |

教学规则 没有反证列的候选表，通常只是模型把第一种解释写得更完整。

## 证据质量：来源、时间和覆盖决定可用性



- ▶ 旧文档、采样 Trace 和延迟指标都可能是真数据，但不一定适合当前事故；
- ▶ Evidence Bundle 对每条证据附带 quality 标签；
- ▶ 当质量不足以支持结论时，Agent 应输出“证据不足”，而不是降低语言强度继续猜。

## Context Budget: 先保留能改变决策的内容

### 优先保留

- ▶ 当前症状与对照；
- ▶ 能区分候选的关键点；
- ▶ 证据来源、时间和质量；
- ▶ 已排除的假设。

### 可压缩或延后

- ▶ 重复日志模板；
- ▶ 与当前范围无关的历史；
- ▶ 已经被证伪的长解释；
- ▶ 无来源的自然语言摘要。

**工程结论** 上下文管理不是 Prompt 技巧，而是证据选择、成本控制和可复核性的共同问题。



---

# RCA Agent 的最小代码骨架（伪代码）

## 伪代码

```
run = init_task(envelope)
while not terminal(run):
    plan = planner.next(run.state, run.evidence)
    checked = policy.check(plan)
    result = tool.invoke(checked)
    run = verifier.update(run, result)
    trace.append(plan, result, run.state)
return renderer.render(run) or handoff(run)
```

- ▶ ‘planner’ 不能绕过 ‘policy’;
- ▶ ‘verifier’ 不能只检查 JSON 合法，还要检查证据和状态;
- ▶ ‘terminal’ 必须同时考虑成功、失败、预算和人工接管。

# 三、诊断循环：候选、信息增益与反证

让 Agent 每一步都能说明 “为什么查这个、查完改变了什么”

# ReAct 不够：RCA 需要证据驱动的循环

## ReAct 的启发

- ▶ Reason：形成当前判断；
- ▶ Act：调用工具；
- ▶ Observe：读取返回。

## RCA 的额外要求

- ▶ 记录候选集合和置信变化；
- ▶ 对支持、反证、未知分别建模；
- ▶ 在验证或接管前不把候选写成事实；
- ▶ 能重放、比较、注入和审计。

推理文本可以被压缩或隐藏，但证据状态不能丢。

---

## Hypothesis Store: 候选是有生命周期的对象

| 字段                     | 示例                           | 作用            |
|------------------------|------------------------------|---------------|
| <b>id / label</b>      | H1 / network path            | 可引用和跨轮比较      |
| <b>belief</b>          | low / medium / high          | 候选排序, 不等同概率校准 |
| <b>support</b>         | span-42、change-7             | 绑定原始证据        |
| <b>counterevidence</b> | endpoint healthy             | 防止只收集支持       |
| <b>next_test</b>       | cross-zone trace             | 选择下一条查询       |
| <b>status</b>          | open / eliminated / verified | 状态转移和终态       |

注意 belief 是调查中的工作分数；只有在定义清楚标签、对照和验证后，才可以讨论准确率或概率校准。

---

## 候选排序：分数来自证据，不来自语言强度

### 教学抽象

$$S(h) = \alpha \textit{Support}(h) - \beta \textit{Contradict}(h) + \gamma \textit{Prior}(h) + \delta \textit{Fresh}(h) - \lambda \textit{Risk}(h)$$

- ▶ ‘Support’：观测、拓扑、变更和历史是否支持；
- ▶ ‘Contradict’：是否存在时间、对照、范围或终态矛盾；
- ▶ ‘Fresh’：证据新鲜度、覆盖和来源质量；
- ▶ ‘Risk’：若采取验证动作，影响面、成本和回滚难度。

公式不是为了伪装精确，而是强迫系统把排序理由结构化。

---

## 信息增益：下一条查询应该改变候选分布

### 教学直觉

$$IG(q) = H(Hypotheses) - \mathbb{E}_{r \sim q}[H(Hypotheses \mid r)]$$

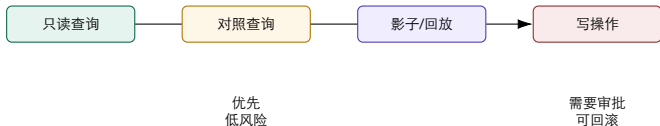
高信息增益 一次查询能明显区分网络、进程和负载候选。

低信息增益 重复看同一指标，只让上下文更长，却不改变决策。

成本单列 信息增益只度量不确定性减少；查询调度可再用

$U(q) = IG(q) - \lambda Cost(q) - \mu Risk(q)$  权衡成本与风险，不能把 Cost 混入 IG 的定义。

## 查询调度：低风险、高区分力、可回放



- ▶ 先查能区分候选的观测，再决定是否需要干预；
- ▶ 干预本身也要设计成实验：前测、动作、后测、停止条件；
- ▶ 没有对照或终态指标的动作，不能证明根因。

## 并行还是串行？取决于依赖关系和风险

### 可以并行

- ▶ 同一时间窗的指标、日志、变更查询；
- ▶ 互不影响的服务依赖检查；
- ▶ 只读且预算可控的证据采集。

### 必须串行

- ▶ 查询结果决定下一条工具参数；
- ▶ 动作前后的对照与回归验证；
- ▶ 有副作用、审批或共享状态的步骤。

**结论** 并行提高吞吐，串行保留因果顺序；Agent Runtime 需要显式表达两类依赖。



---

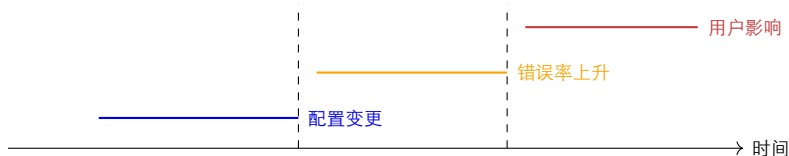
## 反证优先：让模型主动寻找会推翻自己的数据

1. 对每个高分候选生成一条“如果它是真的，应该看到什么”；
2. 再生成一条“什么观察会推翻它”；
3. 查询反证工具，而不是继续搜支持句子；
4. 若反证未查到，状态仍是 unknown，而不是 verified。

### 示例

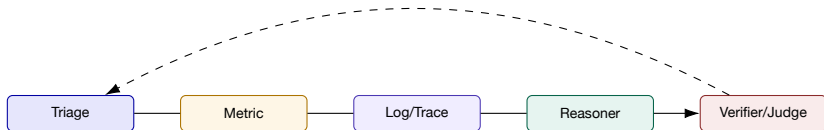
**H1 网络路径**：若成立，跨区 Trace 应失败而同区 Trace 正常；若两者都失败，H1 不能独立解释全局故障。

## 时间对齐：先后顺序是 RCA 的最低门槛



- ▶ 变更晚于症状，只能是修复或响应，不是先因；
- ▶ 数据源的时间水位和时钟漂移要记录；
- ▶ 时间先后仍不等于因果，需要结构、对照或干预。

## 多 Agent 分工：拆能力，不拆责任



- ▶ Triage 负责范围和优先级；Metric/Log 负责证据获取；
- ▶ Reasoner 负责候选和调查计划；Judge 负责一致性、缺口和终态；
- ▶ 最终 owner 仍属于任务，而不是某个 Agent 角色。

# Verifier: 检查格式之外，还要检查证据结构

|               |                  |                 |
|---------------|------------------|-----------------|
| 检查项           | 通过条件             | 失败处理            |
| Schema        | 字段、类型、枚举合法       | 请求重写或接管         |
| Provenance    | 每个关键主张有来源        | 降低结论等级          |
| Consistency   | 时间、拓扑、指标不矛盾      | 生成反证查询          |
| Coverage      | 关键服务、窗口和模态覆盖     | 标记 evidence gap |
| Action safety | 风险、审批、回滚齐全       | 禁止执行            |
| Terminal      | 业务指标或 oracle 已验证 | 回到调查或 handoff   |

关键结论 Verifier 不是让模型再写一遍答案，而是给诊断结果增加独立的结构、证据和安全检查。

---

## 停止条件：什么时候 **Agent** 应该停？

**Resolved** 候选有足够证据，验证通过，业务恢复。

**Monitor** 候选明确但不能安全干预，进入观察窗口。

**Handoff** 证据冲突、预算耗尽、权限不足或风险过高。

- ▶ “模型还可以继续查”不是继续运行的理由；
- ▶ 每个终态都要记录原因、缺口、责任人和下一步；
- ▶ 失败停机比生成一个错误的确定性答案更可控。

---

## 示例调查时间线：每一步都改变了什么？

| 时刻    | Agent 操作               | 新证据                     | 候选变化            |
|-------|------------------------|-------------------------|-----------------|
| 10:00 | 归一化事件与范围               | 503 仅跨区出现               | H1 上升, H2/H3 保留 |
| 10:01 | 查 Endpoint / readiness | Endpoint 有效、进程健康        | H2 降低, H1 上升    |
| 10:03 | 查跨区 Trace              | proxy 到 api 的边界 span 失败 | H1 获得结构支持       |
| 10:05 | 查策略与注入对象               | 当前窗口存在网络扰动对象            | H1 获得时间与对象支持    |
| 10:07 | 只读验证与审批请求              | 删除对象需要负责人确认             | 进入 PROPOSE/HITL |
| 10:10 | 删除后回归检查                | 503 恢复, 跨区 span 正常      | H1 得到干预支持       |

这是课程教学重绘，用于演示结构化轨迹，不是生产统计。

## 示例运行结果：从候选到结构化 RCA

**H1**

网络路径 / 注入扰动，最终验证支持

**5**

只读查询

**1**

人工审批动作

- ▶ Evidence coverage: 关键主张均绑定指标、Trace、拓扑或变更来源；
- ▶ Counterevidence: 后端 readiness 和同区直连正常，降低进程故障候选；
- ▶ Action: 只对故障注入对象执行最小删除，不重启全服务；
- ▶ Verify: 跨区请求恢复、错误率回落、Trace 边界消失。

**结果结论** 在这个教学场景中，Agent 的有效性来自“查询顺序 + 证据矩阵 + 最小干预 + 回归验证”，不是来自一句更长的解释。

# 示例结论的边界：验证支持不等于普遍因果

|                                                            |                                                          |                                                               |
|------------------------------------------------------------|----------------------------------------------------------|---------------------------------------------------------------|
| 可以说<br>该窗口内删除扰动后跨区请求恢复<br>后端健康指标降低了进程故障可能性<br>最小动作通过业务回归验证 | 不能直接说<br>所有 503 都由网络扰动导致<br><br>后端永远不是根因<br>删除动作在任何环境都安全 | 还需要什么<br>跨事故、跨版本和对照样本<br><br>更细粒度进程、资源和业务证据<br>影响面、权限、回滚和灰度策略 |
|------------------------------------------------------------|----------------------------------------------------------|---------------------------------------------------------------|

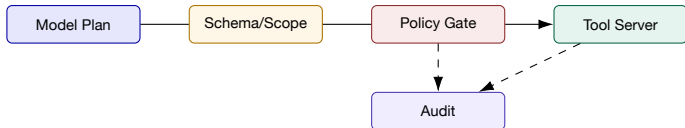
研究生要求 RCA 结论必须写清适用范围、证据强度和下一步，而不是只输出一个根因名词。



## 四、安全闭环：审批、回滚与记忆

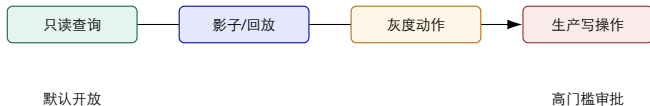
把“建议”与“执行”拆开，保证失败可停止、可恢复、可审计

## 工具门禁：模型不能绕过 Policy Gate



- ▶ 模型只提出计划，Policy Gate 决定是否允许；
- ▶ 任何写操作必须带审批、幂等键、影响面、回滚和终态检查；
- ▶ 规则变化和策略命中记录进 Trace，支持审计和复盘。

## 只读、影子、灰度、生产：四级动作权限



Agent 的自主等级应由动作风险、可逆性和影响面决定，而不是由模型置信度单独决定。

---

## 审批卡：人要审的是风险和证据，不是按钮

|      |                         |              |
|------|-------------------------|--------------|
| 审批字段 | 示例                      | 为什么需要        |
| 动作   | 删除故障注入对象                | 明确实际副作用      |
| 理由   | H1 得到 Trace + 时间 + 回归支持 | 把结论绑定证据      |
| 影响面  | 单一测试命名空间                | 判断风险和范围      |
| 回滚   | 恢复对象版本 / 重新启用规则         | 防止不可逆        |
| 验证   | 503、跨区 Trace、业务探针       | 定义成功而非只看命令返回 |
| 审批   | owner、时间、过期时间           | 责任和审计        |

---

**HITL 结论** 人不是最后“按允许”的人，而是确认证据、风险、回滚和终态条件的人。

## ActionStack: 动作、回滚、验证必须成组



- ▶ Pre-check 不通过，Action 不得执行；
- ▶ Health 失败，进入停止或回滚；
- ▶ Rollback 成功也要验证业务结果、告警和 Trace，而不是只看命令返回码。

---

## 终态 Oracle: Agent 要知道“什么叫恢复”

服务层 错误率、延迟、  
readiness、队列、资源。

调用层 关键路径、重试、  
超时、依赖边界。

业务层 成功率、订单、  
答案质量、用户探针。

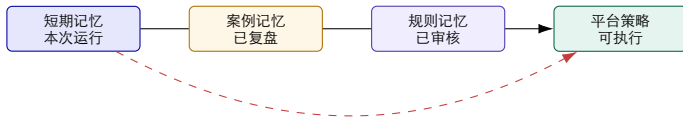
- ▶ 终态 Oracle 可以是规则、指标窗口、回放结果或人工确认；
- ▶ 不同故障需要不同 Oracle，不能用“模型说已恢复”替代；
- ▶ Oracle 的定义应在动作前冻结，避免事后挑选有利指标。

# 安全策略表： 风险随动作和不确定性共同变化

| 动作      | 不确定性低   | 不确定性高     | 默认策略    |
|---------|---------|-----------|---------|
| 读指标/日志  | 允许      | 允许但标记缺口   | 自动      |
| 生成报告    | 允许      | 必须引用证据    | 自动 + 抽检 |
| 修改测试对象  | 单人确认    | 双人确认 + 灰度 | 审批      |
| 修改生产配置  | 双人/负责人  | 禁止自动      | 严格审批    |
| 重启/删除资源 | 双人 + 回滚 | 禁止自动      | 人工接管    |

工程结论    高置信度不能抵消高风险；低置信度也不能阻止低风险的只读验证。

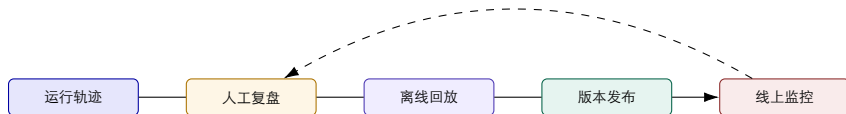
## 记忆分层：事故轨迹不能直接变成永久规则



- ▶ 短期记忆记录观察、假设和待办；
- ▶ 案例记忆经过人工复盘，包含证据、动作和结果；
- ▶ 规则记忆必须有版本、适用范围、过期时间和回滚路径。



## 规则晋升：从事故经验到可复用知识的门槛



一次错误结论不能直接成为下一次事故的高优先级提示；离线阶段是风险隔离层。

## 在线与离线：事故响应和系统学习分开

### 在线

- ▶ 快速构造失败上下文；
- ▶ 生成当前候选和验证建议；
- ▶ 延迟、证据完整性和安全优先。

### 离线

- ▶ 回放历史轨迹和失败样本；
- ▶ 评估规则、Prompt、工具和模型版本；
- ▶ 把人工确认结果按版本回流。

**WeRCA** 的工程启示 在线负责当下事故，离线负责上下文构造、规则演化和低置信度案例处理。

# 五、工作台、论文结果与故障注入

把抽象 Agent 变成能看、能跑、能测的系统

## 工作台：先看系统，再问 Agent



拓扑、服务、依赖和健康状态先提供“当前世界”的上下文，再进入对话和诊断。

来源：课程案例界面独立裁切；教学重绘与标注。

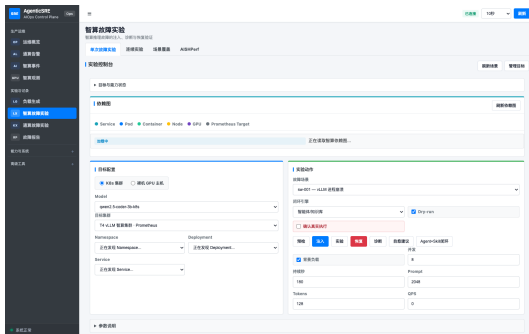
# Assistant: 让人看到问题、证据和下一步



对话框应与证据、工具调用、候选和审批状态并列，而不是只展示自然语言。

来源：课程案例界面独立裁切；教学重绘与标注。

# Workflow: 把一次对话变成可检查的过程

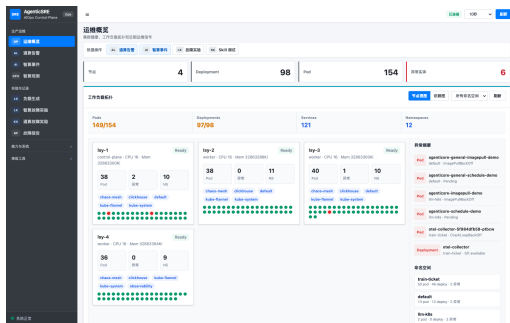


- ▶ 节点是步骤、工具、判断或审批点；
- ▶ 连线表达条件、数据依赖和错误分支；
- ▶ 运行后可以回到每个节点查看输入、输出和耗时。

来源：课程案例 workflow 界面独立裁切；教学重绘。



# Hypothesis 实验：让每次诊断都可比较



- ▶ 固定事件、窗口、工具版本和终态指标；
- ▶ 比较不同查询顺序、模型、规则和验证器；
- ▶ 将结果回写同一条 Trace，支持回放和回归。

来源：课程案例实验界面独立裁切；教学重绘。



# 观测面板：不是看 Agent 说了多少话



过程 工具、状态、耗时、重试。

证据 覆盖、来源、新鲜度、冲突。

结果 任务、业务、成本、安全和接管。

来源：课程案例观测界面独立裁切；教学重绘。

# WeRCA: 在线诊断与离线知识演化

RCA framework.

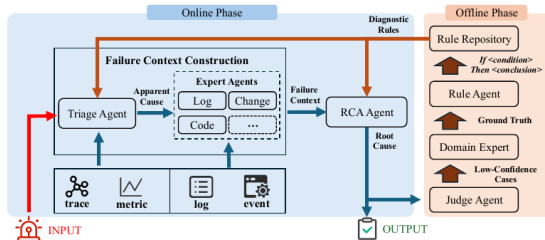


Fig. 1. The overview of WeRCA.

- ▶ 在线阶段通过 Triage、专家 Agent 和 RCA Agent 构造失败上下文；
- ▶ 离线阶段用规则、领域知识和评审处理历史案例；
- ▶ 课堂实现启示：先把“在线诊断”和“规则晋升”拆成两个状态域。

来源：WeRCA 独立论文图；课堂按在线/离线职责重绘说明。

## WeRCA 的结果与边界：数字要带条件

**71.43%**

公开材料中的条件化场景结果

**2**

主要评估系统数量

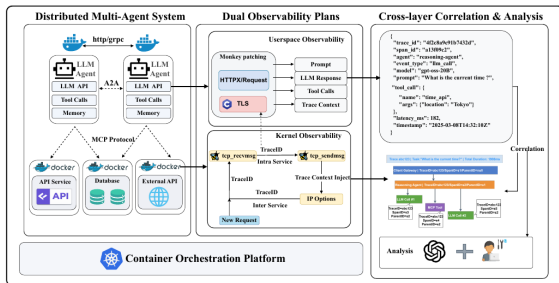
**在线 + 离线**

规则演化闭环

- ▶ 结果支持“失败上下文构造 + Agent 协作 + 规则回流”这条系统思路；
- ▶ 结果不等于任意企业平台上的通用准确率；
- ▶ 规则覆盖、模型波动、系统差异和人工确认仍是外部有效性边界。

来源：WeRCA 论文/作者材料；数字保留原实验条件，不作生产 SLA。

# AgentTracer: 把 Agent 调查变成可观测执行轨迹



- ▶ Root Task 下面展开模型、工具、环境和结果节点；
- ▶ 每个节点携带输入、输出、耗时、错误和上下文版本；
- ▶ 对 RCA 而言，Trace 是重放、评测、回归和审计的共同底座。

来源：AgentTracer 独立论文图；课堂按 RCA 任务标注。

---

# Trace 结果怎么读？覆盖、顺序、结果缺一不可

## Coverage

关键工具和状态是否出现

## Order

是否遵守前置、审批和验证顺序

## Outcome

是否完成任务并改善终态

- ▶ 有完整 Trace，不代表 Agent 得出了正确诊断；
- ▶ 没有工具和终态节点，任何“成功”都难以复核；
- ▶ 平台应同时记录成功轨迹和失败轨迹，避免只保留漂亮案例。

## LLMRCA: 多模态观测如何进入层次化验证?

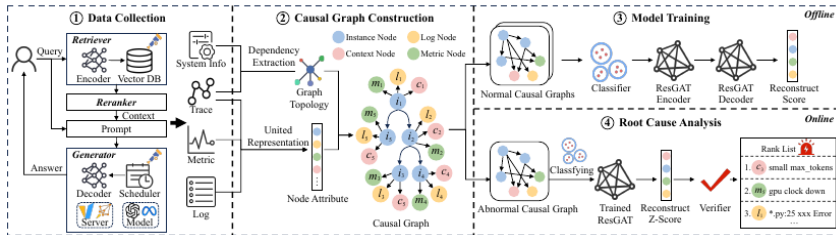


Fig. 7. The framework of LLMRCA.

LLMRCA 将 instance、metric、log、context 和质量故障组织成多层诊断问题；RCA Agent 可借鉴“先粗粒度，再细粒度验证”。

来源：Tan et al., LLMRCA, ACM TOSEM 2025；独立论文图。

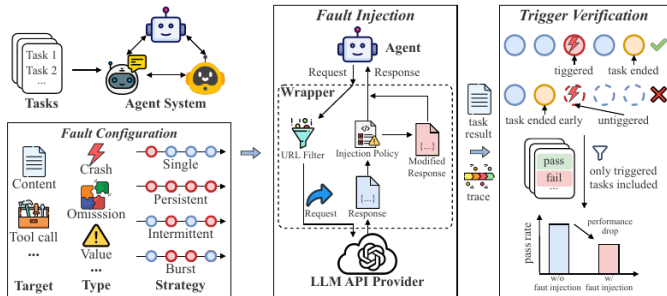
## LLMRCA 结果：三类任务分别读

| 任务                 | HR@1   | NDCG@3 |
|--------------------|--------|--------|
| AP component-level | 84.23% | 89.20% |
| AP inner-component | 73.81% | 81.99% |
| RQ root cause      | 92.86% | 97.36% |

- ▶ 实验在一个 RAG 应用中注入 42 个故障、覆盖 10 类；
- ▶ 组件级结果和根因级结果不能互换解释；
- ▶ 对 RCA Agent 的启示：层次化检索和 verifier 能缩小搜索空间，但仍依赖观测覆盖与标签定义。

来源：Tan et al., LLMRCA, ACM TOSEM 2025；论文受控实验。

# AgentChaos: 把 Agent 当作需要做可靠性测试的系统



- ▶ 注入响应截断、字段遗漏、值破坏、超时和异常返回；
- ▶ 观察任务成功、故障类型诊断和故障步骤诊断；
- ▶ 将注入点映射到工具、状态、模型和终态 **Trace**。

来源：Tan et al., AgentChaos, ASE 2026 / arXiv:2608.06790；独立论文图。



## AgentChaos 结果：最危险的是“坏了但诊不出”

**50 pp**

pass@1 最大下降

**<53%**

故障类型诊断

**<56%**

故障步骤诊断

- ▶ 所测系统在故障注入下均出现任务性能退化；
- ▶ 故障类型和故障步骤诊断能力低于任务成功率；
- ▶ 结论：RCA Agent 评测必须包含“为什么失败”，而不是只测“是否完成”。

来源：Tan et al., AgentChaos, ASE 2026 / arXiv:2608.06790；受控实验，不作生产外推。

# OpsLens: 多角色调查如何协作?

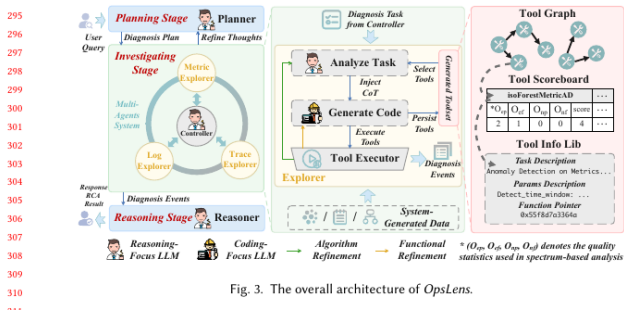


Fig. 3. The overall architecture of OpsLens.

- ▶ 不同角色负责不同证据视角，避免一个 Agent 同时承担搜索、判断和执行；
- ▶ 共享任务上下文、结果缓存和证据引用；
- ▶ 汇总角色负责冲突处理、终态和人工接管。

来源: OpsLens 论文材料独立图; 课堂按工具与证据职责重绘。

# OpsLens 结果如何解释?

**0.80**

公开材料中的 OpenRCA score rate

**多角色**

证据协作

**条件化**

任务、数据和系统绑定

- ▶ 结果说明多角色证据协作有潜力，但指标定义决定含义；
- ▶ 不能把一个 **score rate** 直接翻译成生产根因准确率；
- ▶ 工程上仍要测工具命中、证据覆盖、终态验证和人工接受度。

来源：OpsLens 论文/作者材料；数字为条件化结果。

# 六、综合案例：从 503 到可验证的根因结论

用同一条 Agent Trace 串起证据、审批、动作与恢复

## 案例：跨边界请求 503，但后端看起来健康

### 症状

- ▶ frontend 到 api 的 503 增加；
- ▶ api readiness 正常；
- ▶ 同区直连成功，跨区请求失败。

### 初始候选

1. 网络路径/策略；
2. Endpoint 或服务发现；
3. 后端进程/资源；
4. 流量、队列或版本变更。

**第一步** 只读查拓扑、Endpoint、跨边界 Trace、指标和变更，暂不重启或改生产路由。

---

## 案例输入：Task Envelope

---

| 字段                 | 值                                          |
|--------------------|--------------------------------------------|
| <b>incident</b>    | inc-demo-503                               |
| <b>scope</b>       | tenant=training, service=api, cluster=demo |
| <b>window</b>      | 10:00-10:20, 含前后对照窗口                       |
| <b>permissions</b> | read-only; 动作需要负责人审批                       |
| <b>budget</b>      | 8 次查询、1 次候选动作、10 分钟运行时限                    |
| <b>done_when</b>   | 候选有证据, 动作可回滚, 业务探针恢复或转人工                   |

---

把范围和预算写进任务对象，能防止 Agent 在事故窗口内无限探索。

## 案例计划：先做能区分候选的查询



- ▶ q1 确认影响面；q2 排除后端实例健康；
- ▶ q3 判断传播路径；q4 绑定时间和对象；
- ▶ q5 只有在证据足够、风险可控并获批后才执行。

# 案例证据表：每次查询都要改变候选状态

| 查询       | 观察                  | 支持/反证     | 状态变化     |
|----------|---------------------|-----------|----------|
| q1 指标    | 503 只在跨区出现          | 支持/反证     | H1 上升    |
| q2 健康    | readiness、进程和同区直连正常 | 支持路径候选    | H2 降低    |
| q3 Trace | 跨区边界 span 超时        | 反证 H2     | H1 高     |
| q4 变更    | 时间窗内存在网络扰动对象        | 支持 H1     | H1 更高    |
| q5 回归    | 删除对象后跨区成功           | 时间 + 对象支持 | verified |
|          |                     | 干预支持 H1   |          |

可复核 任何一条结论都能回到 query\_id、时间水位、来源和原始结果。



## 案例 Agent Trace：工具调用是状态，不是黑盒日志

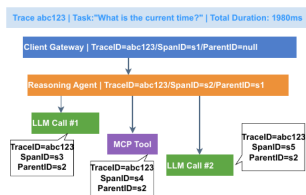


Figure 2: An example trace spanning a 3-service DMAS workflow, illustrating hierarchical span relationships and the causal decomposition of a user query into agent-level execution steps.

- ▶ 每个工具节点带参数摘要、返回状态、耗时、成本和证据引用；
- ▶ 失败调用进入重试或替代路径，不能静默消失；
- ▶ 最终输出绑定整条轨迹，支持“为什么得出这个候选”的回放。

来源：AgentTracer 独立 Trace 图；本页叠加案例标注。

---

## 案例输出：结构化 RCA 结果

### 结论

候选：网络路径/扰动对象。      强度：verified in this window。      动作：删除故障对象，单一命名空间，已审批。

支持    跨区 Trace、时间先后、对象存在、删除后恢复。

反证    后端 readiness、同区直连和进程指标正常。

缺口    未覆盖其他区域、版本和长期回归。

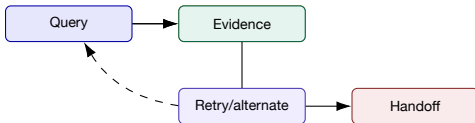
下一步    保留回放轨迹，提交案例复盘；只有经过离线验证和人工审核，才考虑晋升为规则。

# 案例审批与验证：动作前就冻结成功条件

|    |                   |                |
|----|-------------------|----------------|
| 项目 | 动作前               | 动作后            |
| 范围 | 单一测试命名空间          | 检查实际对象和副本      |
| 风险 | 影响面、回滚、过期时间       | 观察异常扩大和旁路影响    |
| 验证 | 503、跨区 Trace、业务探针 | 5 分钟稳定窗口、错误率回落 |
| 责任 | owner、审批人、接管人     | 结果确认和复盘链接      |

成功条件在动作前冻结，避免动作后只挑选一个有利指标来宣布恢复。

## 失败分支：如果工具超时或证据冲突怎么办？



- ▶ 重试有次数和预算上限；替代工具需声明语义差异；
- ▶ 证据冲突时进入 UNKNOWN/HANDOFF，而不是选一个顺耳答案；
- ▶ 人工接管携带当前候选、缺口、轨迹和建议下一步。

---

## 故障注入实验：验证 Agent 是否真的能诊断失败

工具层 空结果、错误  
schema、超时、权限拒绝。

状态层 循环转移、重复  
调查、预算耗尽、上下文  
丢失。

终态层 错误恢复、响应  
截断、业务未恢复但文本  
宣布成功。

- ▶ 每次注入都记录 `fault_id`、注入位置、预期症状和 `oracle`；
- ▶ 比较 `baseline Agent`、加 `verifier`、加 `evidence bundle` 和加终态 `oracle`；
- ▶ 结果不仅看任务成功率，还看故障定位、步骤定位、证据覆盖和安全接管。

## 实验设计：先写假设，再跑 Agent

| 假设                              | 对照         | 处理                                | 结果指标                   |
|---------------------------------|------------|-----------------------------------|------------------------|
| H1: <b>Verifier</b> 减少无证据结论     | 无 verifier | schema + provenance + consistency | groundedness、错误结论率     |
| H2: <b>Evidence Bundle</b> 缩短调查 | 全量 Prompt  | 可排序证据包                            | 工具次数、Token、时间          |
| H3: 终态 <b>Oracle</b> 降低假成功      | 文本判定       | 业务/服务 oracle                      | false resolved、handoff |
| H4: 信息增益减少重复查询                  | 固定顺序       | query scheduler                   | 重复率、候选收敛               |

实验纪律 一次运行中的结果不能同时被用来选择工具、调 Prompt 和定义成功标准；否则会造成评估泄漏。

---

## 评测指标：五个维度一起看

### Task

是否完成任务

### Source

主张是否有来源

### Tool

参数和命中是否正  
确

### Safety

是否越权或误执行

### Cost

Token、延迟和人工

- ▶ 任务成功但证据缺失，是“看似完成”；
- ▶ 证据完整但终态失败，是“解释正确但没有解决”；
- ▶ 成本低但误执行率高，不是可接受的优化。

---

## 结果矩阵：把数字翻译成工程结论

| 结果观察              | 可以支持的结论       | 不能直接支持的结论             |
|-------------------|---------------|-----------------------|
| HR@1 上升           | 候选排序在该任务定义上更好 | 生产根因一定正确              |
| 工具次数下降            | 查询规划或缓存更有效    | 调查质量没有下降              |
| false resolved 下降 | 终态 oracle 有价值 | 所有业务恢复都能被当前 oracle 覆盖 |
| 成本上升              | 证据/模型/验证更昂贵   | 高成本一定换来高可靠性           |
| 人工接管上升            | 系统更谨慎或工具更差    | Agent 没有价值            |

---

结论写法 先说测量结果，再说在什么条件下支持什么设计，最后写清外推边界。



# 本讲方法的结果与结论总表

|            |                              |                      |             |
|------------|------------------------------|----------------------|-------------|
| 方法         | 结果/证据                        | 工程结论                 | 边界          |
| WeRCA      | 条件化场景 71.43%，在线 + 离线         | 失败上下文与规则演化要分层        | 系统和数据集绑定    |
| LLMRCA     | 三类任务 HR@1 73.81–92.86%       | 层次验证缩小多模态搜索          | 单一应用、注入故障   |
| AgentChaos | pass@1 最大降 50 pp，诊断低于 53/56% | 必须测 “为什么失败”          | 受控注入与系统配置绑定 |
| 课程示例       | 5 次只读查询 + 1 次审批 + 回归恢复       | 查询顺序、反证和 oracle 形成闭环 | 教学重绘，非生产统计  |

这张表是本讲的核心结论页：方法结果、平台设计和有效性边界必须同时出现。

# 七、课堂研讨与交付

把 RCA Agent 变成可以被同学复现、质疑和改进的实验对象

---

## 课堂练习：为一个新故障写 Task Envelope

1. 症状：TTFT 上升、TPOT 稳定、KV Cache 接近上限；
2. 候选：长上下文、队列堆积、GPU 降频、网络传输；
3. 写出范围、时间窗、权限、预算和 done\_when；
4. 选择三条低风险查询，并为每条写预期区分信号；
5. 规定什么时候进入 PROPOSE、VERIFY 或 HANDOFF。

讨论 如果没有业务探针，什么可以作为终态 Oracle？它的盲区是什么？

---

## 课堂练习：设计一次 AgentChaos 注入

|        |                          |                   |
|--------|--------------------------|-------------------|
| 设计项    | 示例                       | 学生需要回答            |
| 注入点    | log.search 返回 partial    | Agent 是否标记证据缺口？   |
| 预期症状   | 候选排序不稳定                  | 哪些 Trace 字段能定位？   |
| Oracle | 不允许 false resolved       | 业务/服务结果如何判定？      |
| 对照     | 无 verifier vs 有 verifier | 哪个指标证明改进？         |
| 安全     | 不得触发写操作                  | Policy Gate 如何拦截？ |

故障注入不是为了让系统“看起来很脆弱”，而是为了暴露不可诊断、不可接管的路径。

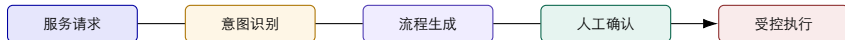
---

## 项目交付建议：一份 RCA Agent 设计说明书

1. 任务合同：输入、输出、状态、预算和停止；
2. 工具目录：schema、权限、失败语义和审计；
3. Agent Loop：计划、证据、候选、反证和 verifier；
4. 安全闭环：审批、回滚、终态 Oracle 和接管；
5. 实验报告：数据、对照、结果、解释和边界；
6. 回放材料：至少一条成功轨迹和一条失败轨迹。

**评分重点** 不是 Prompt 写得长，而是合同完整、结果可复核、失败可解释、动作可控。

## 第 14 讲衔接：从 RCA Agent 到自然语言 ITSM 流程



- ▶ RCA Agent 解决“如何调查和验证”；
- ▶ ITSM Agent 还要解决“如何把意图转成流程和工单”；
- ▶ 本讲的 Task Envelope、工具契约、审批和 Oracle 会继续复用。

---

## 参考材料与论文

1. WeRCA: Multi-modal Failure Context Construction and Iterative Enhancement for Root Cause Analysis, 2026;
2. Tan et al., *LLMRCA: Multilevel Root Cause Analysis for LLM Applications Using Multimodal Observability Data*, ACM TOSEM 2025;
3. Tan et al., *AgentChaos: Chaos Engineering for Agent Systems via Programmatic Fault Injection*, ASE 2026 / arXiv:2608.06790;
4. AgentTracer 论文材料: Agent/Tool/Observation/Outcome Trace;
5. OpsLens 论文材料: 多角色运维调查与证据协作;
6. Yao et al., *ReAct*, ICLR 2023; StateFlow; TapeAgents; MCP/tooling materials;
7. OpenTelemetry Semantic Conventions for Generative AI; Langfuse Observability documentation;
8. 'agentworkflows.pdf'、'ai-agents.pdf'、'tooling.pdf'、'ai-infra-engineer-learning-main';
9. 本讲图片均为独立论文图、独立界面裁切或 LaTeX/TikZ 教学重绘, 不使用完整第三方 PPT 页面。

论文数字只在对应实验条件下解释; 课程案例轨迹只用于说明实现与验证方法。

---

## 课后反思

1. 你的 Agent 如何证明“查对了”，而不是只证明“回答了”？
2. 哪一种工具失败最容易导致错误根因？如何让 Trace 显式暴露它？
3. 为什么终态 Oracle 必须在动作前定义？
4. 论文结果、课程案例和你自己的实验，分别能支持多强的结论？

**开放问题** 可靠 RCA Agent 的终点不是一句漂亮的根因描述，而是证据、动作、恢复、接管和复盘可以连成一条链。



# 谢谢

从任务合同到可验证的 RCA Agent